

SIMULATED STOCK EXCHANGE

System Design & Architecture Whitepaper

A Real-Time, Scalable, Event-Driven Financial Trading System

Version 1.0 | March 2026

Confidential | Internal Use

1. Executive Summary

This whitepaper presents the complete system design for a Simulated Stock Exchange — a high-concurrency, real-time financial trading platform built as a production-grade portfolio project. The system mimics the core operations of a real-world stock exchange: users authenticate securely, place buy and sell orders, watch a live order book update in real time, and receive instant trade confirmations.

The platform is designed to be horizontally scalable, fault-tolerant, and event-driven. It incorporates OAuth 2.0 + JWT authentication, WebSocket-based live data streaming, an in-memory order matching engine backed by Redis, asynchronous job processing via BullMQ, and PostgreSQL for persistent ACID-compliant storage. Automated trading bots continuously simulate market activity, ensuring the system is always under realistic load.

This document covers the system architecture, each component's design rationale, data models, API contracts, the matching engine algorithm, scalability strategies, and a phased implementation timeline.

2. Project Overview

2.1 What Is This System?

Think of this project as a miniature version of NASDAQ or the Bombay Stock Exchange — but built from scratch for educational and portfolio purposes. Users can:

- Register and log in securely using OAuth 2.0 (Google/GitHub) or email/password with JWT tokens.
- View a live order book that shows the current best buy prices (bids) and sell prices (asks) for each stock.
- Place market orders (buy/sell at whatever price is available) or limit orders (buy/sell only at a specific price or better).
- Receive instant notifications when their orders are matched and trades are executed.
- Track their portfolio: which stocks they hold, how much they paid, and their current profit/loss.

Behind the scenes, the system runs automated trading bots that place randomised buy and sell orders, simulating realistic market activity 24/7.

2.2 Why Build This?

Financial systems are among the most demanding in software engineering. They require:

- Low latency — orders must be matched in milliseconds.
- High concurrency — thousands of users placing orders simultaneously.
- Data consistency — no double-spending, no lost orders, no phantom trades.
- Real-time communication — all users see the same order book simultaneously.

Building this system demonstrates mastery of distributed systems, real-time communication, event-driven architecture, and financial-grade data integrity — all of which are highly valued by engineering teams.

3. System Architecture

3.1 High-Level Architecture Overview

The system is composed of four main layers that work together:

Layer	Components	Purpose
Client Layer	React Frontend, Mobile Browsers	User interface for trading, portfolio view, live order book
API Gateway Layer	Node.js + Express, OAuth Provider	Auth, order submission, REST APIs, rate limiting
Processing Layer	Matching Engine, BullMQ Workers, Bot Service	Core trading logic, async jobs, simulated market activity
Data Layer	PostgreSQL, Redis	Persistent storage, real-time cache, pub/sub messaging

3.2 Request Flow: Order Placement

Here is what happens, step-by-step, when a user places a buy order:

1. User submits a POST /api/orders request with their JWT token in the header.
2. The API server validates the JWT, checks the idempotency key (prevents duplicate orders), and validates the order parameters.
3. The order is written to PostgreSQL with status 'open', and simultaneously pushed into a BullMQ queue called 'order-matching'.
4. A BullMQ worker picks up the job and sends it to the Matching Engine.
5. The Matching Engine checks Redis for existing sell orders. If a matching sell order exists, a trade is executed.
6. On match: both orders are updated to 'matched' in PostgreSQL, user balances and portfolios are updated, and a 'TradeExecuted' event is emitted.
7. Redis Pub/Sub broadcasts the updated order book snapshot to all WebSocket servers.
8. All connected clients receive the live update via Socket.IO within milliseconds.

9. A 'trade-settlement' job is queued in BullMQ to send email confirmations and update portfolio recalculations asynchronously.

3.3 Architecture Diagram (Described)

The following describes the system's component relationships:

- React Frontend connects to Node.js API via HTTPS/REST and to Socket.IO servers via WebSocket.
- Node.js API Servers (multiple instances, load-balanced) connect to PostgreSQL, Redis, and BullMQ.
- BullMQ Workers (independently scalable) run the Matching Engine and settlement jobs.
- Redis acts as the central nervous system: order book cache, session store, rate-limit counters, and Pub/Sub message broker.
- Bot Service is a separate Node.js process that generates simulated orders at configurable rates.
- PostgreSQL is the single source of truth for all persistent data.

4. Authentication & Authorisation

4.1 OAuth 2.0 Flow

OAuth 2.0 is an industry-standard protocol that lets users log in using a trusted third-party provider (like Google or GitHub) without sharing their password with your application. Here is how it works in simple terms:

10. The user clicks 'Login with Google' on the frontend.
11. They are redirected to Google's login page. They log in there, not on your site.
12. Google sends back an authorization code to your server's callback URL.
13. Your server exchanges that code for an access token (and optionally a refresh token) directly with Google.
14. Your server looks up or creates the user in your database, then issues your own JWT tokens.

This means you never store the user's Google password. You just trust Google to verify who they are.

4.2 JWT Token Strategy

Once OAuth is complete (or a user logs in with email/password), the system issues two tokens:

Token	Lifetime	Purpose	Storage
Access Token	15 minutes	Authenticate every API request	Memory / HTTP-only cookie
Refresh Token	7 days (rotated)	Get a new access token silently	HTTP-only secure cookie

Asymmetric signing means the server signs tokens with a private RSA key, and any service can verify them using only the public key — without needing access to the secret. This is critical in a multi-server environment.

Refresh token rotation means every time the frontend uses the refresh token to get a new access token, the old refresh token is invalidated and a brand new one is issued. If an attacker steals a refresh token and uses it, the legitimate user's next request will fail, alerting the system to a potential breach.

4.3 OTP for Sensitive Actions

For high-stakes actions like withdrawals or password changes, the system sends a 6-digit One-Time Password (OTP) via email. The OTP is stored in Redis with a 10-minute expiry. This adds a second factor of verification without requiring a full 2FA setup.

5. Live Order Book

5.1 What Is an Order Book?

An order book is a live list of all pending buy and sell orders for a given stock, sorted by price. The 'bids' side shows buyers: the highest prices people are willing to pay. The 'asks' side shows sellers: the lowest prices people are willing to accept. The gap between the highest bid and the lowest ask is called the 'spread'.

Example Order Book for AAPL:

Side	Price (INR)	Quantity	Number of Orders
ASK (Sell)	153.50	200	3
ASK (Sell)	153.00	500	7
--- SPREAD ---	---	---	---
BID (Buy)	152.90	300	4
BID (Buy)	152.50	800	12

In this example, the best ask is 153.00 and the best bid is 152.90. A market buy order would immediately match against the 153.00 ask.

5.2 Storing the Order Book in Redis

Redis Sorted Sets are perfect for order books. Each stock has two sorted sets:

- BIDS:AAPL — sorted by price descending (highest bids first). Score = price.
- ASKS:AAPL — sorted by price ascending (lowest asks first). Score = price.

Each entry in the sorted set maps to an order ID, which can be looked up in a Redis Hash for full order details. This gives $O(\log N)$ insertion and retrieval — extremely fast even with millions of orders.

When the matching engine executes a trade, it updates these sorted sets atomically. Redis Pub/Sub then broadcasts the updated order book snapshot to all Node.js WebSocket servers, which push the update to every connected browser in real time.

5.3 WebSocket Architecture

Socket.IO is used for the WebSocket layer. When a user opens the trading dashboard:

15. Their browser establishes a WebSocket connection to the nearest Node.js instance.
16. They subscribe to the order book channel for the stocks they care about.
17. Any time the order book changes (a new order, a match, a cancellation), Redis Pub/Sub broadcasts the delta.
18. All Node.js instances receive this broadcast and forward it to their connected clients.

This design ensures that even if you run 10 Node.js instances behind a load balancer, all users see the same live data simultaneously.

6. Order Matching Engine

6.1 The Core Algorithm: Price-Time Priority

The matching engine is the heart of the exchange. Its job is simple to describe but complex to implement correctly: find buy and sell orders that can be paired together and execute a trade.

The algorithm uses Price-Time Priority (also called FIFO matching):

- Price Priority: A higher-priced buy order always gets matched first. A lower-priced sell order always gets matched first.
- Time Priority: If two orders have the same price, the one placed earlier gets matched first. This rewards fast traders and prevents fairness issues.

6.1.1 Market Orders

A market order says 'buy/sell immediately at whatever price is available'. The engine takes the best available opposite-side order (lowest ask for a buy, highest bid for a sell) and executes the trade at that price. If the quantity is larger than one order, it continues matching against the next best orders until fully filled.

6.1.2 Limit Orders

A limit order says 'buy at this price or lower / sell at this price or higher'. The engine first checks if there is a matching order. If yes, it trades immediately. If no, the order is added to the order book and waits.

6.2 Matching Engine Logic (Step by Step)

When a new BUY limit order arrives for AAPL at price 153.00, quantity 100:

19. Retrieve the best ask from Redis: the lowest-priced sell order currently in the book.
20. If best ask price $\leq$ 153.00: there is a match. Proceed to step 3. Otherwise: add the buy order to the bid side of the order book and stop.
21. Calculate trade quantity = $\min(\text{buy_quantity}, \text{sell_quantity})$.
22. Execute the trade: deduct quantity from both orders, update balances.
23. If either order is fully filled, remove it from the order book. If partially filled, update the remaining quantity.
24. Repeat from step 1 until the buy order is fully filled or no more matching asks exist.

6.3 Concurrency & Atomicity

Multiple BullMQ workers run simultaneously, so two workers could theoretically try to match against the same sell order at the same time. This is solved with Redis Lua scripts — atomic operations that check and update the order book in a single, indivisible step. No two workers can modify the same order simultaneously.

Additionally, when writing trade results to PostgreSQL, the system uses database transactions with row-level locking on user balance rows, preventing the 'double-spend' problem where a user could accidentally buy more stock than their balance allows.

7. Background Processing with BullMQ

7.1 Why Use a Job Queue?

Not every operation needs to happen immediately when a user clicks a button. Sending an email, recalculating a portfolio, or running daily summaries can all happen in the background — without making the user wait. BullMQ is a job queue built on Redis that lets you define 'jobs' and process them asynchronously with multiple workers.

7.2 Queue Architecture

Queue Name	Jobs It Handles	Processing Time
order-matching	Match incoming orders against the order book	< 50ms (critical path)
trade-settlement	Update portfolios, deduct balances, confirm trades	< 200ms
notifications	Send trade confirmation emails, push notifications	1-5 seconds
scheduled-jobs	Market open/close events, daily P&L summaries, bot activity	Repeatable
order-expiry	Cancel unfulfilled limit orders after market close	Delayed jobs

7.3 Key BullMQ Features Used

- Job Priorities: Order matching jobs run at the highest priority. Email jobs run at low priority.
- Retries with Backoff: If a job fails (e.g., database timeout), BullMQ retries it with exponential backoff — wait 1s, then 2s, then 4s, etc.
- Repeatable Jobs: Market open at 9:15 AM and close at 3:30 PM IST are scheduled using BullMQ's cron-style repeatable jobs.
- Delayed Jobs: When a limit order is placed, a delayed job is scheduled to cancel it if it has not been matched by market close.
- Dead Letter Queue: Jobs that fail repeatedly are moved to a dead letter queue for manual inspection.

8. Data Models

8.1 PostgreSQL Schema

Users Table

Column	Type	Description
id	UUID (PK)	Unique user identifier

Column	Type	Description
email	VARCHAR UNIQUE	User email address
password_hash	VARCHAR	Bcrypt hash (null for OAuth users)
oauth_provider	VARCHAR	google / github / null
oauth_id	VARCHAR	Provider's unique ID for this user
balance	DECIMAL(15,2)	Cash balance available for trading
created_at	TIMESTAMP	Account creation time

Orders Table

Column	Type	Description
id	UUID (PK)	Unique order ID
user_id	UUID (FK)	References users.id
stock_symbol	VARCHAR	e.g., AAPL, RELIANCE, INFY
order_type	ENUM	'market' or 'limit'
side	ENUM	'buy' or 'sell'
price	DECIMAL(15,2)	Limit price (null for market orders)
quantity	INTEGER	Total quantity ordered
filled_quantity	INTEGER	Quantity matched so far
status	ENUM	'open', 'partially_filled', 'filled', 'cancelled'
idempotency_key	VARCHAR UNIQUE	Prevents duplicate submissions
created_at	TIMESTAMP	Order placement time

Trades Table

Column	Type	Description
id	UUID (PK)	Unique trade ID
buy_order_id	UUID (FK)	References orders.id
sell_order_id	UUID (FK)	References orders.id
stock_symbol	VARCHAR	Stock traded
price	DECIMAL(15,2)	Price at which trade executed
quantity	INTEGER	Quantity traded
executed_at	TIMESTAMP	Exact time of trade execution

Portfolio Holdings Table

Column	Type	Description
user_id	UUID (FK)	References users.id
stock_symbol	VARCHAR	Stock held
quantity	INTEGER	Number of shares held
avg_buy_price	DECIMAL(15,2)	Average cost basis
updated_at	TIMESTAMP	Last updated time

8.2 Redis Data Structures

Key Pattern	Redis Type	Contents
BIDS:{SYMBOL}	Sorted Set	Order IDs scored by bid price (desc)
ASKS:{SYMBOL}	Sorted Set	Order IDs scored by ask price (asc)
ORDER:{ORDER_ID}	Hash	Full order details: price, qty, user_id
SESSION:{TOKEN}	String + TTL	User session data, expires in 15min
RATE:{USER_ID}	Counter + TTL	Request count for rate limiting
OTP:{USER_ID}	String + TTL	6-digit OTP, expires in 10min
ORDERBOOK_SNAPSHOT:{SYMBOL}	JSON String	Latest full order book snapshot

9. Scalability Strategy

9.1 Horizontal Scaling

The system is designed to scale horizontally — meaning you can add more servers as traffic grows, without redesigning the architecture. Here is how each layer scales:

Component	Scaling Strategy	What It Achieves
Node.js API	Run multiple instances behind an Nginx load balancer	Handles more HTTP requests per second
WebSocket Servers	Multiple Socket.IO servers sharing state via Redis Adapter	Supports more concurrent WebSocket connections
BullMQ Workers	Spin up more worker processes/containers	Processes more orders per second

Component	Scaling Strategy	What It Achieves
Redis	Redis Cluster with sharding	Distributes memory and throughput
PostgreSQL	Read replicas for queries; primary for writes	Reduces read load on primary DB
Bot Service	Run multiple bot instances with configurable intensity	Simulates more realistic market volume

9.2 Rate Limiting

Each user is limited to 100 API requests per minute, enforced using Redis counters. The key is `RATE:{user_id}`, incremented on each request with a 60-second TTL. If the counter exceeds 100, the request is rejected with HTTP 429 Too Many Requests. This prevents a single user (or bot) from overwhelming the system.

9.3 Database Performance

- Connection Pooling: Use pg-pool to maintain a pool of database connections, avoiding the overhead of creating a new connection for every request.
- Indexes: Create composite indexes on `(user_id, status)` for order queries and `(stock_symbol, side, price)` for matching queries.
- Partitioning: For very high volume, partition the trades table by month so that historical queries do not scan the entire table.
- Row-Level Locking: Use `SELECT ... FOR UPDATE` when reading user balances before a trade to prevent race conditions.

9.4 Caching Strategy

User portfolio data is cached in Redis for 30 seconds. The order book snapshot is always served from Redis, not PostgreSQL — making reads nearly instantaneous. Only writes (new orders, matched trades) touch the database.

10. Trading Bot Architecture

10.1 Purpose of Bots

Without real users, the order book would be empty and the system would be static. Trading bots solve this by continuously placing realistic buy and sell orders across all available stocks. They keep the system lively, stress-test the matching engine, and make the portfolio project more impressive to demonstrate.

10.2 Bot Behaviour

Each bot instance follows a configurable strategy:

Strategy	Description	Use Case
Random Market Maker	Places both bid and ask orders within a spread around the last trade price	Maintains a healthy order book
Trend Follower	Buys when price is rising, sells when falling	Creates price momentum
Mean Reverter	Buys when price dips below average, sells when above	Stabilises extreme price swings
Volume Bot	Places large orders at random intervals	Stress-tests the matching engine

10.3 Bot Implementation

The Bot Service is a standalone Node.js process (separate from the API server). It:

- Runs on a configurable tick interval (e.g., every 500ms).
- Generates orders with slightly randomised prices around a 'fair value' that drifts over time.
- Submits orders via the same REST API as real users (authenticated with a bot JWT token).
- Has configurable aggression: how often it places market orders vs limit orders.
- Can simulate N simultaneous bots by running multiple async loops.

Because bots use the same API as users, they automatically exercise the full system stack: auth, rate limiting, matching, WebSocket broadcasts, and settlement.

10.4 Bot Configuration

Parameter	Default	Description
BOT_COUNT	10	Number of simultaneous bot instances
TICK_INTERVAL_MS	500	How often each bot places an order (milliseconds)
PRICE_VARIANCE	0.5%	How far from fair value a bot will bid/ask
MARKET_ORDER_PROBABILITY	0.2	Probability of placing a market order vs limit (0-1)
ENABLED_STOCKS	['AAPL','RELIANCE','INFY']	Stocks the bots trade

11. Fault Tolerance & Data Integrity

11.1 Idempotency

Idempotency means that sending the same request twice produces the same result as sending it once — no duplicate orders, no double charges. Every order submission requires a unique `idempotency_key` (a UUID generated by the frontend). The server stores this key in the database with a unique constraint. If the same key arrives again (e.g., due to a network retry), the server returns the original response without creating a new order.

11.2 Event Sourcing for Crash Recovery

All order state changes (`OrderPlaced`, `OrderMatched`, `OrderCancelled`, `TradeExecuted`) are written as immutable events to an events table in PostgreSQL. If a server crashes mid-processing, it can replay these events to reconstruct the exact state of the order book and all in-flight orders. Nothing is lost.

11.3 Graceful Shutdown

When a Node.js process receives a `SIGTERM` signal (e.g., during a deployment), it:

25. Stops accepting new HTTP requests.
26. Waits for all in-flight requests to complete (up to 30 seconds).
27. Closes database connections cleanly.
28. Signals BullMQ workers to finish their current job before exiting.

This prevents orders from being lost during routine deployments.

11.4 Redis Persistence

Redis is configured with both RDB snapshots (periodic disk dumps) and AOF (Append-Only File) logging. This means that even if Redis crashes, it can recover the order book state from disk, rather than starting empty.

12. API Reference

12.1 Authentication Endpoints

Method	Endpoint	Description
POST	<code>/api/auth/register</code>	Register with email + password
POST	<code>/api/auth/login</code>	Login, returns access + refresh tokens
GET	<code>/api/auth/google</code>	Initiate Google OAuth flow

Method	Endpoint	Description
GET	/api/auth/google/callback	OAuth callback, issues JWT tokens
POST	/api/auth/refresh	Refresh access token using refresh token
POST	/api/auth/logout	Invalidate refresh token
POST	/api/auth/otp/send	Send OTP for sensitive action
POST	/api/auth/otp/verify	Verify submitted OTP

12.2 Order Endpoints

Method	Endpoint	Description
POST	/api/orders	Place a new order (idempotency key required)
GET	/api/orders	List user's orders (with filters: status, symbol)
GET	/api/orders/:id	Get single order by ID
DELETE	/api/orders/:id	Cancel an open order

12.3 Market Data Endpoints

Method	Endpoint	Description
GET	/api/orderbook/:symbol	Get current order book snapshot for a stock
GET	/api/trades/:symbol	Get recent trade history (tape) for a stock
GET	/api/stocks	List all available stocks with last price

12.4 Portfolio Endpoints

Method	Endpoint	Description
GET	/api/portfolio	Get user's current holdings and cash balance
GET	/api/portfolio/history	Get trade history for the user

13. Technology Stack

Technology	Version	Role in System
Node.js	v20 LTS	Runtime for API servers, workers, and bot service

Technology	Version	Role in System
Express.js	v4	HTTP routing, middleware, REST API framework
Socket.IO	v4	WebSocket management, real-time order book broadcasting
Redis	v7	Order book cache, sessions, rate limiting, Pub/Sub
BullMQ	v4	Job queue for async order processing, scheduled tasks
PostgreSQL	v16	Persistent storage: users, orders, trades, portfolios
Passport.js	v0.6	OAuth 2.0 strategy middleware (Google, GitHub)
jsonwebtoken	v9	JWT signing and verification (RSA asymmetric)
React	v18	Frontend trading dashboard
Docker + K8s	Latest	Containerisation and orchestration for deployment
Nginx	v1.25	Load balancer, reverse proxy, SSL termination

14. Implementation Timeline

Phase 1 — Foundation (Weeks 1–2)

Goal: Get authentication, basic API, and database running.

- Set up project structure: monorepo with /api, /workers, /bots, /frontend directories.
- Configure Docker Compose with Node.js, PostgreSQL, and Redis containers.
- Implement user registration and email/password login with JWT (access + refresh tokens).
- Integrate Passport.js for Google OAuth 2.0 flow.
- Create users, orders, trades, and portfolio_holdings tables in PostgreSQL.
- Build basic order submission endpoint (POST /api/orders) with idempotency key support.
- Write automated tests for auth flows.

Deliverable: A working API where users can register, log in via Google or email, and submit an order that gets saved to the database.

Phase 2 — Matching Engine (Weeks 3–4)

Goal: Orders get matched and trades execute.

- Implement Redis sorted set order book (BIDS and ASKS for each stock).
- Build the price-time priority matching algorithm in a standalone module.
- Integrate BullMQ: order-matching queue + worker.
- Connect order submission endpoint to BullMQ queue instead of direct processing.

- Implement trade settlement: update balances and portfolio holdings in a PostgreSQL transaction.
- Write unit tests for the matching engine with edge cases (partial fills, market orders, empty book).

Deliverable: Users can place orders, and the system matches compatible buy/sell orders, executing trades and updating portfolios.

Phase 3 — Real-Time Layer (Week 5)

Goal: Live order book and trade notifications via WebSockets.

- Set up Socket.IO server with room-based subscriptions (one room per stock symbol).
- Configure Redis Pub/Sub: publish order book updates after every match.
- Subscribe Socket.IO servers to Redis channels and broadcast to clients.
- Implement GET /api/orderbook/:symbol endpoint for initial snapshot on page load.
- Add trade tape endpoint (GET /api/trades/:symbol) for recent execution history.

Deliverable: The frontend receives live order book updates in real time without polling.

Phase 4 — Bots & Market Simulation (Week 6)

Goal: A lively, realistic market with automated traders.

- Build the Bot Service as a separate Node.js process.
- Implement Random Market Maker strategy as the default bot behaviour.
- Add configurable parameters: bot count, tick interval, price variance.
- Connect bots to the API using dedicated bot accounts with JWT auth.
- Add at least one additional strategy (e.g., Trend Follower).
- Stress-test the matching engine with 10 bots running simultaneously.

Deliverable: The exchange is always active, with bots providing continuous liquidity.

Phase 5 — Notifications & Scheduled Jobs (Week 7)

Goal: Complete the async processing layer.

- Add 'notifications' BullMQ queue and email worker (using Nodemailer or SendGrid).
- Send trade confirmation emails when orders are matched.
- Implement BullMQ repeatable jobs for market open and close events.
- Add delayed jobs to cancel unfulfilled limit orders at market close.
- Implement OTP flow for sensitive user actions.
- Build daily P&L summary job for each user's portfolio.

Deliverable: Users receive emails on trade execution; market events run on schedule.

Phase 6 — Frontend Dashboard (Weeks 8–9)

Goal: A polished, interactive trading UI.

- Build React app with pages: Login, Dashboard (order book + charts), Portfolio, Orders.
- Integrate Socket.IO client for live order book updates.
- Implement order entry form with market/limit toggle, quantity, and price fields.
- Display real-time trade tape (scrolling list of recent executions).
- Show portfolio holdings with live P&L calculated from current prices.
- Add basic candlestick chart for price history (use lightweight-charts library).

Deliverable: A fully functional trading dashboard that non-technical users can demo.

Phase 7 — Hardening & Deployment (Weeks 10–12)

Goal: Production-ready, deployable, demonstrable system.

- Add rate limiting middleware using Redis counters.
- Implement graceful shutdown handling for all services.
- Configure Redis with RDB + AOF persistence.
- Set up Docker Compose for the full system (API, workers, bots, frontend, Redis, PostgreSQL).
- Write a Kubernetes deployment configuration for multi-instance scaling demo.
- Add monitoring: basic health check endpoints (/health), Redis memory stats.
- Write comprehensive README with architecture overview and setup instructions.
- Deploy to a cloud provider (Railway, Render, or AWS EC2) with a live demo URL.

Deliverable: A fully deployed, live system accessible via a public URL, with a video walkthrough.

15. Project Timeline Summary

Phase	Weeks	Key Milestone
1 — Foundation	1–2	Auth working, database running, basic order API
2 — Matching Engine	3–4	Buy/sell orders matched, trades executed
3 — Real-Time Layer	5	Live order book via WebSockets
4 — Bot Service	6	Automated market simulation active
5 — Notifications	7	Emails, scheduled jobs, OTP
6 — Frontend	8–9	Full trading dashboard UI
7 — Hardening & Deploy	10–12	Live deployment, K8s, monitoring

16. Conclusion

The Simulated Stock Exchange is a comprehensive, production-grade system that demonstrates expertise across the full spectrum of modern backend engineering. It addresses the hardest problems in distributed systems: real-time data synchronisation, concurrent order processing, data consistency under high load, and fault-tolerant asynchronous workflows.

By incorporating OAuth 2.0, JWT authentication, WebSocket broadcasting, Redis-backed order books, BullMQ job queues, and a PostgreSQL persistence layer — all wired together with an event-driven architecture — this project serves as a tangible proof of engineering capability.

The addition of automated trading bots ensures that the system is always under realistic load, making live demonstrations compelling. The phased 12-week timeline breaks a complex system into manageable milestones, each delivering working, testable software.

Any engineering team that reviews this project will see evidence of the ability to architect, build, and deploy a system that would be equally at home in a fintech startup or a large financial institution.

End of Whitepaper — Simulated Stock Exchange v1.0